

Introdução

Conceitos básicos e motivação

Prof. Marcelo de Souza

45RPE – Resolução de Problemas com Estruturas de Dados
Universidade do Estado de Santa Catarina



Algumas definições:

- ▶ **Algoritmo:** sequência de passos para realizar uma tarefa.
- ▶ **Estrutura de dados:** forma sistemática de **organizar** e **acessar** os dados.

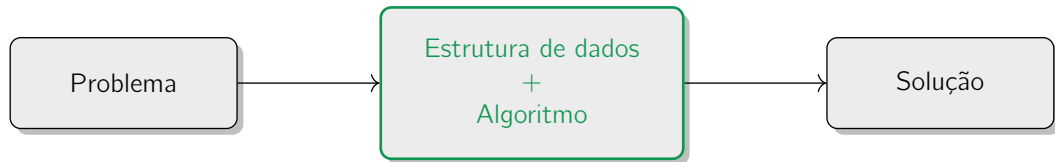


Algumas definições:

- ▶ **Algoritmo:** sequência de passos para realizar uma tarefa.
- ▶ **Estrutura de dados:** forma sistemática de **organizar** e **acessar** os dados.

Objetivo: escolher os melhores elementos para resolver um problema. Ou seja, buscando

- ▶ eficácia: resolve o problema;
- ▶ eficiência: usa a menor quantidade de recursos possível.





Para isso, precisamos **conhecer**/**dominar**:

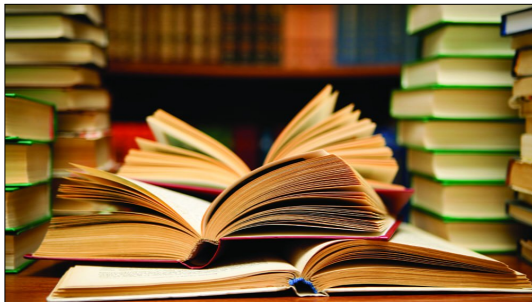
- ▶ alguma linguagem de programação (nesta disciplina, Python);
- ▶ orientação a objetos;
- ▶ análise da complexidade de algoritmos e classes de complexidade;
- ▶ estruturas de dados fundamentais (*arrays*, listas encadeadas, ...);
- ▶ estruturas abstratas de dados (filas, pilhas, dicionários, árvores, grafos, ...);
- ▶ principais algoritmos que operam sobre essas estruturas;
- ▶ técnicas algorítmicas (recursividade, divisão e conquista, ...);
- ▶ experiência de aplicações práticas a problemas concretos.

Exemplo prático

Palavra mais frequente (*words*)



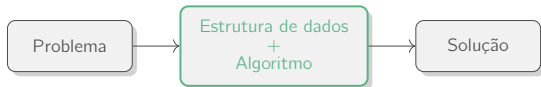
Problema: temos um conjunto de livros e queremos saber qual a palavra mais frequente.



Tarefa: contar as ocorrências de cada palavra e verificar aquela que mais aparece no texto.

Exemplo prático

Palavra mais frequente (*words*)

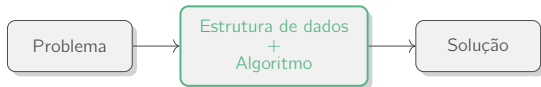


Proposta 1

Usar um vetor para armazenar as palavras encontradas e outro vetor para as ocorrências.

Exemplo prático

Palavra mais frequente (*words*)



Proposta 1

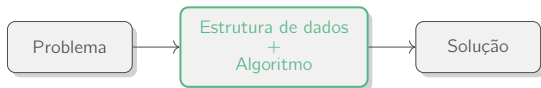
Usar um vetor para armazenar as palavras encontradas e outro vetor para as ocorrências.

Proposta 2

Usar um dicionário para armazenar tanto as palavras quanto as ocorrências.

Exemplo prático

Palavra mais frequente (*words*)



Proposta 1

Usar um vetor para armazenar as palavras encontradas e outro vetor para as ocorrências.

Proposta 2

Usar um dicionário para armazenar tanto as palavras quanto as ocorrências.

Desempenho (grande impacto pela escolha de estruturas de dados e algoritmos!)

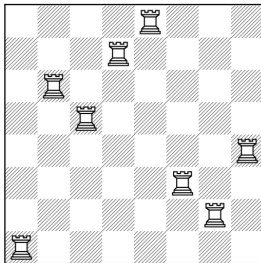
# livros	P. 1	P. 2
1	4,74 s	0,04 s
2	37,85 s	0,08 s
3	112,13 s	0,09 s
4	144,85 s	0,13 s
5	348,82 s	0,14 s

Apêndices

Apêndice I

Problema das n torres (n-rooks)

Problema: posicionar n torres em um tabuleiro de xadrez $n \times n$, de tal forma que elas não se ataquem (a torre se movimenta na vertical e na horizontal).



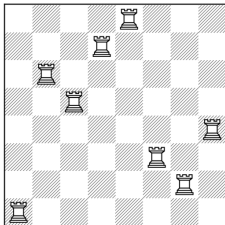
Tarefa: dada uma possível solução (posição das torres no tabuleiro), checar quantas violações existem, i.e. quantos ataques ocorrem.

Apêndice I

Proposta 1 (matriz)

Representar a solução como uma matriz de inteiros $M = (m_{ij})$ de tamanho $n \times n$, tal que $m_{ij} = 1$ caso haja uma torre posicionada na célula (i, j) , e $m_{ij} = 0$, caso contrário.

Exemplo:



```
int[] [] board = {  
    {0, 0, 0, 0, 1, 0, 0, 0},  
    {0, 0, 0, 1, 0, 0, 0, 0},  
    {0, 1, 0, 0, 0, 0, 0, 0},  
    {0, 0, 1, 0, 0, 0, 0, 0},  
    {0, 0, 0, 0, 0, 0, 0, 1},  
    {0, 0, 0, 0, 0, 1, 0, 0},  
    {0, 0, 0, 0, 0, 0, 1, 0},  
    {1, 0, 0, 0, 0, 0, 0, 0}};
```

Implementar um script `nrooks` que faça a leitura de várias soluções, com diferentes tamanhos de tabuleiro, e conte o número de violações (função `violations`).

Apêndice I

Proposta 1 (matriz)

O arquivo `inputMatrix.txt` contém 1000 soluções (uma por linha). Primeiro é fornecido o tamanho $n \in [8, 500]$, seguido do valor de cada célula da matriz:

[illegible]

Apêndice I

Proposta 1 (matriz)



Leitura das soluções e chamada ao método violations:

```
1  def eval_matrix():
2      with open(sys.argv[2], 'r') as file:
3          lines = file.readlines()
4          for line in lines:
5              content = line.strip().split()
6              n = int(content[0])
7              board = [[0] * n for _ in range(n)]
8              next_index = 1
9              for i in range(n):
10                 for j in range(n):
11                     board[i][j] = int(content[next_index])
12                     next_index += 1
13                 print(f'{{violations_matrix(board)}} ', end='')
```



Apêndice I

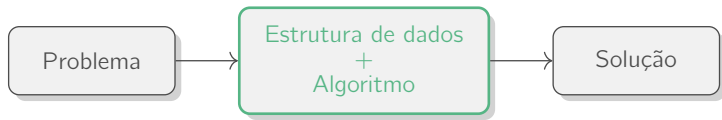
Proposta 1 (matriz)

O método `violations` conta o número de ataques nas linhas e colunas. Seja m o número de torres em uma mesma linha/coluna, o número de ataques é dado por $m(m - 1)/2$:

```
1  def violations_matrix(board):
2      amount = 0
3      for c1 in range(len(board)):
4          found_row = 0
5          found_col = 0
6          for c2 in range(len(board[c1])):
7              if board[c1][c2] == 1:
8                  found_row += 1
9              if board[c2][c1] == 1:
10                 found_col += 1
11             amount += (found_row * (found_row - 1)) / 2
12             amount += (found_col * (found_col - 1)) / 2
13     return amount
```

Apêndice I

Proposta 1 (matriz)

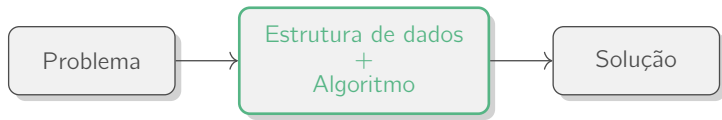


Quão boa é nossa solução para o problema (estrutura de dados + algoritmo)?

- ▶ Tempo de execução: ~**30 s**;
- ▶ Espaço de armazenamento: ~**184 MB** (inputMatrix.txt).

Apêndice I

Proposta 1 (matriz)



Quão boa é nossa solução para o problema (estrutura de dados + algoritmo)?

- ▶ Tempo de execução: ~**30 s**;
- ▶ Espaço de armazenamento: ~**184 MB** (inputMatrix.txt).

Podemos fazer melhor?

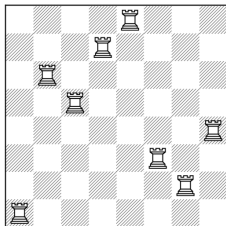
Apêndice I

Proposta 2 (array)



Sim! Assumimos que cada torre ficará em uma linha diferente, e usamos um *array* $A = (a_i)$ de tamanho n , tal que $a_i \in [n]$ indica a coluna onde a torre da linha i estará posicionada.

Exemplo:



→ `int[] board = {4, 3, 1, 2, 7, 5, 6, 0};`

Com isso, temos uma representação mais simples e com menor possibilidade de violações (não permite torres na mesma linha). Essa estrutura admite algoritmos mais eficientes?

Apêndice I

Proposta 2 (*array*)



O arquivo `inputArray.txt` contém as mesmas 1000 soluções anteriores (uma por linha). Primeiro é fornecido o tamanho $n \in [8, 500]$, seguido do valor de cada posição do *array*:

```
1 360 193 278 299 190 81 280 138 250 121 347 56 146 214 320 279 254 354 256 217 17 115 348 ...
2 309 65 69 182 258 123 297 101 97 53 128 229 269 119 186 155 233 226 50 80 195 9 36 98 ...
3 168 144 67 155 102 24 81 27 136 8 136 136 62 157 143 69 69 122 150 53 94 87 60 125 126 ...
4 147 44 81 44 57 92 85 96 84 119 15 62 16 113 98 123 57 30 32 141 131 57 41 9 18 103 6 ...
5 298 103 235 93 264 85 279 137 40 74 140 14 214 296 163 27 121 48 40 8 58 85 85 191 215 ...
6 28 25 19 7 17 16 8 4 2 23 7 5 3 5 4 19 18 27 20 16 3 11 12 20 27 3 14 11 25
7 54 47 45 16 35 45 32 3 22 53 53 37 44 36 39 33 14 24 41 29 37 39 18 18 43 33 27 22 18 17 ...
8 39 18 30 34 10 4 29 30 35 2 23 18 23 23 34 4 30 6 13 36 38 17 19 32 11 35 18 18 7 37 21 ...
9 180 177 107 8 48 56 99 17 8 92 24 146 142 157 41 155 98 134 174 166 2 25 114 43 14 104 ...
10 285 191 182 116 129 44 105 257 18 202 172 274 129 40 105 126 258 153 114 70 8 226 157 ...
11 289 102 110 94 200 260 252 172 274 283 102 277 177 116 147 196 81 84 155 215 98 141 217 ...
12 ...
```

Apêndice I

Proposta 2 (*array*)



Leitura das soluções e chamada ao método `violations`:

```
1 def eval_array():
2     with open(sys.argv[2], 'r') as file:
3         lines = file.readlines()
4         for line in lines:
5             board = []
6             content = line.strip().split()
7             for value in content[1:]:
8                 board.append(int(value))
9             print(f'{violations_array(board)} ', end='')
```

Apêndice I

Proposta 2 (*array*)

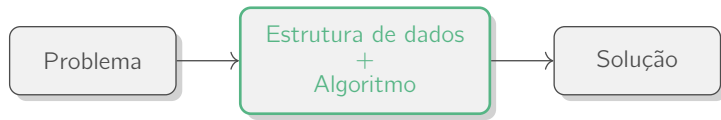


O método `violations` conta o número de valores repetidos no *array*, i.e. o número de torres posicionadas em uma mesma coluna (que corresponde ao número de ataques):

```
1 def violations_array(board):
2     amount = 0
3     for i in range(len(board) - 1):
4         for j in range(i + 1, len(board)):
5             if board[i] == board[j]:
6                 amount += 1
7     return amount
```

Apêndice I

Comparação



Abordagem baseada em **matriz**:

- ▶ Tempo de execução: ~**30 s**;
- ▶ Espaço de armazenamento: ~**184 MB** (`inputMatrix.txt`).

Abordagem baseada em **array**:

- ▶ Tempo de execução: ~**3 s**;
- ▶ Espaço de armazenamento: ~**990 kB** (`inputArray.txt`).

45RPE – Resolução de Problemas com Estruturas de Dados
Prof. Marcelo de Souza